# Webtechnologien – Praktikum 02

## HTML-Grundlagen

## Übungsaufgabe

### Lernziele

- Eine bestehende Seite in semantisches HTML überführen
- Eine Website mit mehreren Seiten, Unterordnern und relativen Links aufbauen
- Formulare erstellen und den Weg der Daten in den DevTools nachvollziehen
- Verstehen, warum Validierung im Browser keine Sicherheit bietet
- Arbeitsstände mit Git sichern und auf GitHub oder GitLab veröffentlichen

Die Aufgabe ist eine unbewertete **Übungsaufgabe**.

### Hinweise zur Abgabe

Es erfolgt keine Abgabe.

### Nutzung von KI-Werkzeugen

Bitte lösen Sie die Aufgaben ohne KI-Werkzeuge wie ChatGPT, Claude oder Copilot. Die Übung ist unbewertet – ihr einziger Zweck ist, dass Sie die Grundlagen selbst verstehen und anwenden. Wenn eine KI den Code schreibt, lernen Sie dabei nichts, und dieses Fundament fehlt Ihnen in den späteren Blöcken.

Deaktivieren Sie dafür auch automatische Code-Vorschläge im Editor, zum Beispiel GitHub Copilot in VS Code. Wenn Sie nicht weiterkommen, fragen Sie Ihre Nachbarn, die Betreuung im Praktikum oder schlagen Sie in den MDN Web Docs nach.

### Vorbereitung

Sie benötigen die Werkzeuge aus Praktikum 1 und zusätzlich:

- GitHub oder GitLab.
- Die Ausgangsdatei `menu.html` aus dem Moodle-Kurs.

### Hinweis zu den Inhalten

Ihre Website muss keine echten persönlichen Daten enthalten. Erfundene Inhalte sind ausdrücklich erlaubt.

## Teil 1: Projekt-Setup

### Aufgabe 1.1: Repository anlegen und klonen

a. Legen Sie auf GitHub oder GitLab über die Weboberfläche ein neues Repository mit dem Namen `praktikum-02` an. Lassen Sie dabei eine README-Datei erzeugen.

b. Klonen Sie das Repository an geeigneter Stelle.

```
git clone <URL Ihres Repositorys>
cd praktikum-02
```

d. Legen Sie den Ordner `public` an, kopieren Sie die Datei `menu.html` aus Moodle hinein und committen Sie diesen Ausgangsstand. Pushen Sie den Commit.

```
git add .
git commit -m "Add original menu page"
git push
```

**Hinweis zum Push:** Je nach Plattform und Einstellung benötigen Sie für Clone und Push ein Personal Access Token oder einen SSH-Schlüssel. Eine Anleitung finden Sie in der Dokumentation von GitHub bzw. GitLab.

## Teil 2: Semantisches HTML

### Aufgabe 2.1: Refactoring der Speisekarte

Die Datei `menu.html` zeigt die Speisekarte eines Campus-Cafés. Sie funktioniert, verwendet aber nur `div`, `b` und `br`.

a. Starten Sie im Ordner `praktikum-02` den Server mit `npx serve public` und öffnen Sie die Speisekarte im Browser.

b. Überführen Sie die Seite in semantisches HTML: einen Seitenrahmen mit `header`, `nav`, `main` und `footer`, jede Kategorie (Drinks, Breakfast, Lunch) als `section` mit Überschrift, die Gerichte als Liste und die Öffnungszeiten mit `time`.

c. Achten Sie auf eine sinnvolle Überschriften-Hierarchie.

d. Vergleichen Sie Ihre Version im Elements-Tab der DevTools mit dem Original. Was hat sich an der Struktur geändert?

e. Committen Sie den Stand:

```
git add .
git commit -m "Refactor menu to semantic HTML"
```

## Teil 3: Eine eigene Website

Bauen Sie eine kleine persönliche Website, zum Beispiel ein Portfolio mit Ihren Projekten oder Hobbys. Quellcode, Datei- und Ordnernamen sind auf Englisch.

```
public/
├─ index.html
├─ menu.html
├─ projects/
│  └─ first-project.html
└─ images/
```

### Aufgabe 3.1: Seiten und Links

a. Erstellen Sie `index.html` mit einer kurzen Vorstellung und einem semantischen Seitenrahmen wie in Teil 2.

b. Erstellen Sie im Ordner `projects` mindestens eine Detailseite.

c. Verlinken Sie die Seiten über eine Navigation in beide Richtungen, auch die Speisekarte aus Teil 2.

### Aufgabe 3.2: Bild und Details

a. Binden Sie auf der Detailseite ein Bild aus dem Ordner `images` ein – mit aussagekräftigem `alt`-Text sowie `width` und `height`.

b. Stellen Sie die Fakten zum Projekt als Definitionsliste (`dl`) dar, zum Beispiel Zeitraum, Werkzeuge und Ihre Rolle.

### Aufgabe 3.3: Links prüfen

a. Klicken Sie sich durch alle Seiten und beobachten Sie dabei das Log des Servers und den Network-Tab. Liefern alle Requests den Status 200?

b. Bauen Sie absichtlich einen fehlerhaften Link ein, zum Beispiel ohne `../`. Was zeigen Log und Network-Tab? Korrigieren Sie den Link anschließend wieder.

c. Committen Sie den Stand:

```
git add .
git commit -m "Add portfolio pages"
```

## Teil 4: Formulare

### Aufgabe 4.1: Kontaktformular

a. Erstellen Sie eine Seite `contact.html` mit einem Formular, das per POST an `/messages` sendet.

b. Das Formular enthält ein Pflichtfeld für den Namen, ein Pflichtfeld für die E-Mail-Adresse (`type="email"`), ein Textfeld für die Nachricht (`textarea`) und einen Button. Jedes Feld hat ein `label`.

c. Testen Sie die Prüfung im Browser: Senden Sie das Formular leer ab und mit einer ungültigen E-Mail-Adresse. Was passiert?

### Aufgabe 4.2: Validierung umgehen

a. Entfernen Sie im Elements-Tab der DevTools das Attribut `required` aus dem Namensfeld und senden Sie das Formular ohne Namen ab. Wird der Request gesendet?

b. Senden Sie einen Request ganz ohne Browser:

```
curl -X POST localhost:8000/messages \
  -d "name=" \
  -d "email=keine-email" \
  -d "message=Test"
```

c. Prüfen Sie im Log des Servers, ob der Request angekommen ist. Was folgt daraus für die Prüfung von Eingaben?

d. Committen Sie den Stand:

```
git add .
git commit -m "Add contact form"
```

## Teil 5: Prüfen und veröffentlichen

### Aufgabe 5.1: Validator

a. Prüfen Sie Ihre Seiten mit dem W3C-Validator (`https://validator.w3.org`). Da der Validator Ihren lokalen Server nicht erreicht, nutzen Sie „Validate by Direct Input“ und fügen den Quelltext ein.

b. Beheben Sie alle Fehler, die der Validator meldet.

### Aufgabe 5.2: Das DOM in der Konsole

a. Öffnen Sie die Console der DevTools auf Ihrer Startseite. Wie viele Links hat die Seite?
`document.querySelectorAll("a").length`

b. Ändern Sie die Hauptüberschrift mit
`document.querySelector("h1").textContent = "Hallo"` und laden Sie die Seite neu. Was passiert, und warum?

### Aufgabe 5.3: Favicon (Zusatzaufgabe)

a. Legen Sie ein kleines Bild als `images/favicon.png` ab und binden Sie es im `head` aller Seiten ein: `<link rel="icon" href="/images/favicon.png">`

b. Warum funktioniert der Pfad mit führendem `/` auch auf den Seiten im Unterordner `projects`?

c. Welche Datei fragt der Browser laut Network-Tab jetzt an?

### Aufgabe 5.4: Veröffentlichen

a. Committen Sie den letzten Stand und pushen Sie alle Commits auf GitHub bzw. GitLab:

```
git add .
git commit -m "Fix validator errors"
git push
```

b. Prüfen Sie in der Web UI, ob alle Commits angekommen sind.

**Checkliste zum Abschluss**

- Alle Seiten sind über die Navigation erreichbar, alle Requests liefern 200.
- Der Validator meldet keine Fehler mehr.
- Ich kann den POST-Request meines Formulars im Network-Tab finden und die Payload lesen.
- Ich kann erklären, warum Python mit 501 und `npx serve` mit 404 antwortet.
- Ich kann erklären, warum der Server jede Eingabe selbst prüfen muss.
- Mein Repository auf GitHub oder GitLab enthält mindestens vier Commits.

**Git-Befehle**

| Befehl | Bedeutung |
|---|---|
| `git clone <URL>` | Repository von GitHub oder GitLab herunterladen |
| `git status` | Geänderte und neue Dateien anzeigen |
| `git diff` | Änderungen seit dem letzten Commit anzeigen |
| `git add .` | Alle Änderungen für den nächsten Commit vormerken |
| `git commit -m "…"` | Vorgemerkte Änderungen mit einer Nachricht sichern |
| `git log --oneline` | Bisherige Commits anzeigen |
| `git push` | Commits auf GitHub oder GitLab hochladen |

**Viel Erfolg!**